Hoisting in JavaScript

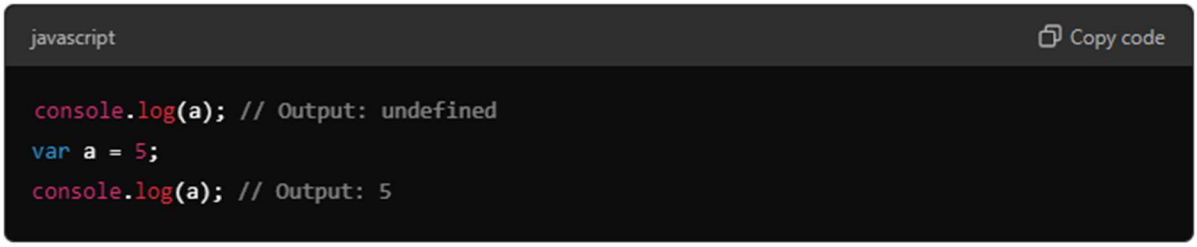
Hoisting is a JavaScript behaviour where variable and function declarations are moved to the top of their scope (global or function scope) during the compile phase, before the code is executed. This means you can use variables and functions before they are actually declared in the code.

However, hoisting only affects the declarations, **not** the initializations (the values assigned to variables).

Hoisting with var

When using var, the variable declaration is hoisted to the top of its scope (function or global scope), but its initialization is not.

Example:




```
console.log(a); // Output: undefined

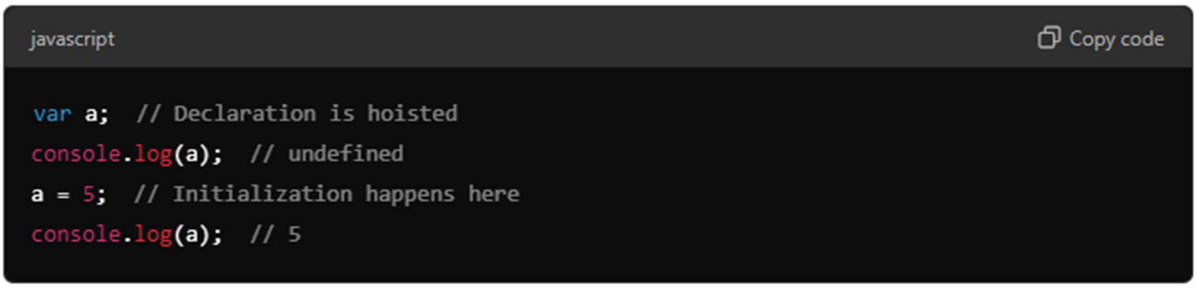
var a = 5;

console.log(a); // Output: 5
```

Explanation:

- The variable a is hoisted, meaning the declaration var a is moved to the top, but the assignment a = 5 remains in place.
- When you try to access a before the assignment, it returns undefined because the declaration is hoisted, but the value isn't yet assigned.

The behaviour can be visualized as:



Hoisting with let and const

Both let and const are also hoisted, but they behave differently compared to var. When using let or const, the variables are placed in a "temporal dead zone" (TDZ) from the start of the block until the declaration is encountered. During this time, accessing the variable results in a ReferenceError.

Example with let:

```javascript
console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 10;
```

Example with const**:**

```javascript
console.log(c); // ReferenceError: Cannot access 'c' before initialization
const c = 15;
```

Explanation:

- The variables b (declared with let) and c (declared with const) are hoisted, but they are not initialized until the code reaches their declaration.
- Trying to access them before their declaration results in a ReferenceError due to the temporal dead zone.

More on temporal dead zone: https://www.freecodecamp.org/news/javascript-temporal-dead-zone-and-hoisting-explained/

Key Differences between var**,** let**, and** const **Hoisting:**

Feature	`var`	`let`	`const`
Hoisted?	Yes (declaration is hoisted)	Yes (but in TDZ until initialization)	Yes (but in TDZ until initialization)
Initialization	Default `undefined` before assignment	No default value before assignment (TDZ)	No default value before assignment (TDZ)
Scope	Function-scoped or globally-scoped	Block-scoped	Block-scoped
Reassignment	Allowed	Allowed	Not allowed (must be initialized at declaration)

More Detailed Examples:

Example with var:

javascript Copy code

```javascript
function varHoisting() {
  console.log(x); // undefined (declaration is hoisted, initialization is not)
  var x = 10;
  console.log(x); // 10
}

varHoisting();
```

Example with let:

javascript Copy code

```javascript
function letHoisting() {
  console.log(y); // ReferenceError: Cannot access 'y' before initialization
  let y = 20;
  console.log(y); // 20
}

letHoisting();
```

Example with const:

```javascript
function constHoisting() {
  console.log(z); // ReferenceError: Cannot access 'z' before initialization
  const z = 30;
  console.log(z); // 30
}

constHoisting();
```

Summary:

- **Hoisting** moves variable and function declarations to the top of their scope. However, var, let, and const behave differently.
- **var** declarations are hoisted and initialized with undefined. This can cause confusing behavior if not handled carefully.
- **let** and **const** are hoisted but placed in a temporal dead zone, making them inaccessible before they are declared. This makes them safer to use because they avoid the pitfalls of unintentional use of undefined before initialization.

~ END ~